Discipline : Enseignement de spécialité NSI

Les arbres binaires de recherche

Contenus : Arbres : structures hiérarchiques. Arbres binaires : nœuds, racines, feuilles, sous-arbres gauches, sous-arbres droits. Récursivité. Algorithmes sur les arbres binaires et sur les arbres binaires de recherche.

Capacités attendues: Identifier des situations nécessitant une structure de données arborescente.
Évaluer quelques mesures des arbres binaires (taille, encadrement de la hauteur, etc.).
Écrire un programme récursif. Analyser le fonctionnement d'un programme récursif.

Calculer la taille et la hauteur d'un arbre. Parcourir un arbre de différentes façons (ordres infixe, préfixe ou suffixe ; ordre en largeur d'abord).

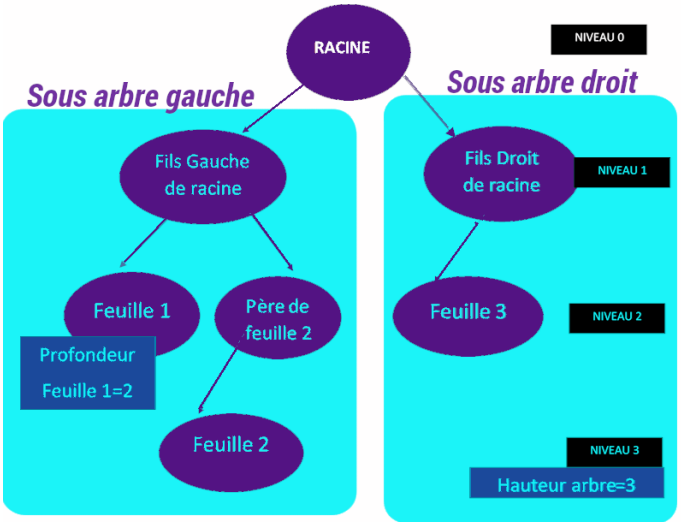
Rechercher une clé dans un arbre de recherche, insérer une clé.

Pré-requis : Langage python, POO

A Arbres

Un arbre est constitué de sommets, que l'on appelle nœuds (ou vertex), et d'arêtes qui relient les sommets entre eux. Ces arêtes sont appelées branches ou arêtes. Pour avoir un arbre, il faut que tous les nœuds soient reliés à un ou deux autres sommets et qu'il n'y ait qu'un seul chemin possible d'un nœud à un autre (il n'y a pas de cycle, c'est à dire, on ne peut pas tourner en rond).

Lorsqu'un arbre a un nœud de départ, duquel partent les premières branches, ce nœud s'appelle la racine de l'arbre. Par la suite nous ne parlerons que d'arbres avec racine.



Si l'on représente l'arbre avec la racine en haut, et en faisant des branches qui descendent, les nœuds issues d'un nœud immédiatement au dessus, s'appellent les fils et le nœud source s'appelle le père (ou fils=enfants=descendants, père=parents). Des nœuds qui ont le même père sont dits frères.

Le nombre de chemins connectés à un nœud s'appelle le degré. Comme dans un arbre il n'y a qu'un seul parent par nœud (sauf pour la racine), degré-1 donne le nombre de fils.

Les nœuds qui ne sont reliés qu'à un seul nœud sont appelés les feuilles (ce sont des nœuds de degré 1, ils n'ont pas de fils). Lorsqu'on représente un arbre verticalement avec la racine en haut, les feuilles sont tout en bas...il suffit de retourner l'arbre pour avoir une analogie parfaite avec les arbres de la nature.

On peut nommer les nœuds. Ces noms sont appelés étiquettes. Lorsque tous les nœuds ont une étiquette, on dit que le graphe est étiqueté. L'étiquette peut indiquer le nom du nœud mais cela peut aussi être une valeur qui donne des informations sur le nœud.
La taille est le nombre de nœuds.

La profondeur ou hauteur d'un arbre est le nombre de branches que contient le chemin le plus long de l'arbre en partant de la racine et en descendant. C'est la distance entre la racine et le nœud. C'est le rang du dernier niveau de l'arbre, la racine étant le niveau 0, ces enfants au niveau 1, etc.
On parle aussi de la profondeur d'un nœud, c'est comme ci-dessus, le niveau où se trouve le nœud (autre façon de le voir, c'est le nombre de branches à utiliser en partant de la racine et en descendant).

On appelle chemin d'un nœud, la suite de nœuds par lesquels il faut passer en partant de la racine.

Exemple de structure en forme d'arbre: Résultats d'un tournoi à élimination directe (tennis par exemple)

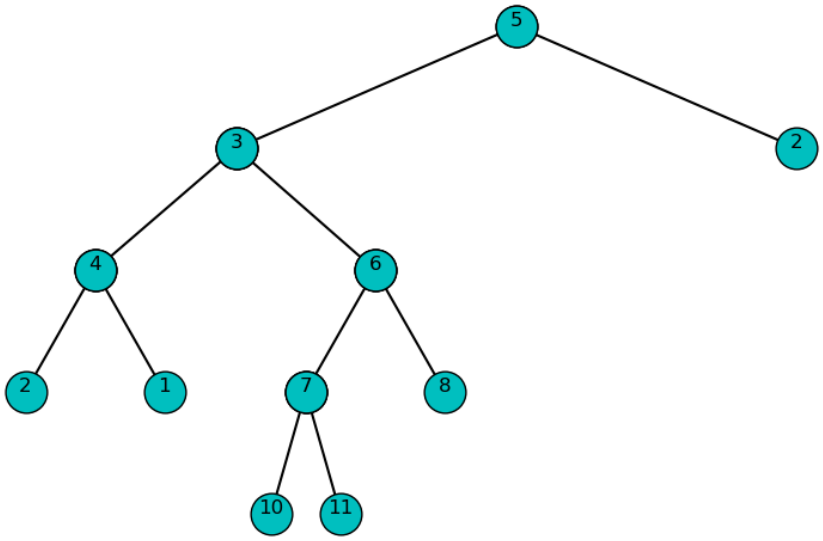
A-2 Arbres binaires

Les arbres binaires (AB) forment une structure de données qui peut être définie récursivement de la manière suivante : un arbre binaire est

- soit vide,
- soit composé d'une racine portant une étiquette (clé) et d'une paire d'arbres binaires, appelés fils gauche et droit.
- Un arbre binaire strict est un arbre dont tous les nœuds ont soit zéro soit deux fils.

Exercice : Déterminer la taille et la hauteur de l'arbre binaire suivant:

arbre= [5,[3,[4,[2,[],[]],[1,[],[]]],[6,[7,[10,[],[]],[11,[],[]]],[8,[],[]]]],[2,[],[]]]
hauteur= 4 taille= 11 nœuds



A-3 Arbre binaire de recherche

La particularité de cet arbre binaire est d'avoir des étiquettes qui sont des nombres entiers appelés clés tels que le fils gauche soit plus petit que le père qui est lui même plus petit que le fils gauche. En regardant l'arbre de gauche à droite on lit une suite de nombres strictement croissante. Un arbre binaire de recherche permet des opérations rapides pour rechercher une clé, insérer ou supprimer une clé.

B Algorithmes sur les arbres binaires et sur les arbres binaires de recherche

B-1 Introduction

Dans cette partie nous allons donner de nombreux algorithmes. Ils seront souvent récursifs. Les algorithmes récursifs sont très puissants mais sont très exigeants, tant dans l'écriture de l'algorithme que dans celle du programme.

Pour pouvoir coder ces algorithmes, il va falloir au préalable arriver à coder un arbre et à en extraire un sous arbre. Dans la suite, on suppose que l'on code les arbres récursivement à partir de nœuds liées les uns aux autres par la relation père/fils. Ainsi, lorsque l'on va calculer la taille d'un arbre, on va plutôt noter taille(nœud_choisi) plutôt que taille(arbre) vu que notre objet est en fait un nœud.

B-2 Calculer la taille et la hauteur

La taille d'un arbre est le nombre de nœuds de l'arbre. L'idée est simple. La taille d'un arbre en partant de la racine est de 1 (pour la racine) plus la taille du sous arbre gauche et celle du sous arbre droit. Le sous arbre gauche a une racine puis on recommence le raisonnement, on est en pleine construction d'un raisonnement récursif.
Algorithme :

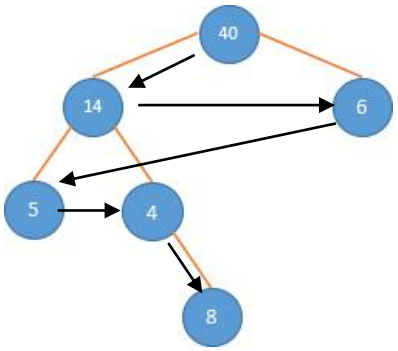
```
fonction taille(arbre):
        si arbre est vide alors
                retourner 0
        sinon
                taille_gauche = taille(sous_arbre_gauche)
                taille_droite = taille(sous_arbre_droit)
                retourner(1+taille_gauche+taille_droite)
```

Pour chercher la hauteur d'un arbre, une méthode est de dire que la hauteur d'un arbre est 1 plus le maximum entre la hauteur du sous arbre gauche et le droit.

```
fonction hauteur(arbre):
        si arbre est vide alors
                retourner -1
        sinon
                hauteur_gauche = hauteur(sous_arbre_gauche)
                hauteur_droite = hauteur(sous_arbre_droite)
                retourner(1+max(hauteur_gauche,hauteur_droite))
```

B-3 Parcours en largeur

Le parcours en largeur (BFS Breadth First Search) d'un arbre est le plus simple. On balaye les nœuds d'un même niveau, avant de passer au niveau suivant. Ici on ne fera pas appel à la récursivité.



Il va falloir visiter les nœuds d'un même niveau et se souvenir qu'ensuite on va descendre voir chacun de leurs enfants qui deviendront eux même un souvenir...

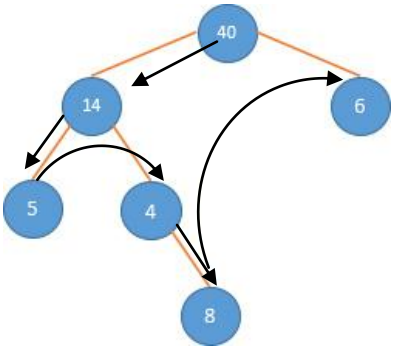
Pour garder en mémoire les nœuds visités dont on ira voir plus tard les enfants, on les places dans une file. Mais attention, il ne faut pas prendre le dernier de la file pour la lecture mais le premier. On est donc en présence d'une file de type FIFO.

```
fonction  parcours_largeur(arbre)
        visite = créer file fifo
        lst=créer liste vide
          ajouter arbre à visite (enfiler)
          tant que visite n'est pas vide
                    nœud reçoit le premier élément de visite (defiler)
                    si nœud est une liste
                              ajouter à la fin de lst le premier élément de nœud
                              si fils droit non vide
                                        ajouter fils droit à visite (enfiler)
                              si fils gauche non vide
                                        ajouter fils gauche à visite (enfiler)
                    sinon
                              ajouter à la fin de lst nœud
```

Résultat pour l'arbre précédent [40, 14, 6, 5, 4, 8]

B-4 Parcours en profondeur

Il s'agit de l'idée inverse de la précédente. Tant que l'on peut descendre, on descend. Dès que l'on ne peut plus, on remonte au nœud parent, que l'on a donc forcément croisé. Il va donc falloir se souvenir des nœuds que l'on n'a pas affichés car on voulait descendre. Dès que l'on remonte à un nœud, on veut afficher ses descendants. On est donc remonté au nœud juste précédent, donc le dernier dont on voulait se souvenir. On est donc en présence d'une pile de type LIFO [last in first of). En changeant la file FIFO de l'algorithme de parcours en largeur par une pile LIFO, on obtient l'algorithme de parcours en profondeur!




```
fonction parcours_profondeur(arbre)
          visite = créer pile lifo
          lst1=créer liste vide
          ajouter arbre à visite (empiler)
          tant que visite n'est pas vide
                    noeud = le dernier élément de visite (dépiler)
                    si nœud est une liste
                              ajouter à la fin de lst1 le premier élément de nœud
                              si fils droit non vide
                                        ajouter fils droit à visite (empiler)
                              si fils gauche non vide
                                        ajouter fils gauche à visite (empiler)
                    sinon
                              ajouter à la fin de lst1 nœud
```

Résultat pour l'arbre précédent [40, 14, 5, 4, 8, 6]

B-5 Ordre de parcours

Il existe plusieurs types de parcours en profondeur. L'exemple ci-dessus correspond à un parcours préfixe. L'algorithme itératif précédent donne l'ordre préfixe.

Dans le parcours préfixe, on lit la racine puis on parcourt récursivement son sous arbre gauche, puis son sous arbre droit.

Dans le parcours infixe, on parcourt récursivement son sous arbre gauche, on lit la racine, puis son sous arbre droit en lisant.

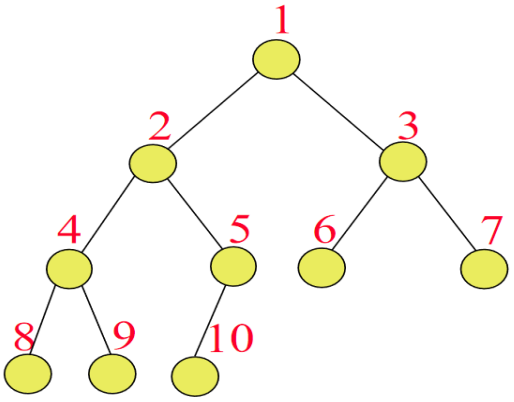
Dans le parcours suffixe, on parcourt récursivement son sous arbre gauche, puis son sous arbre droit en lisant, puis on lit la racine.

Exemple de parcours :

Préfixe : **1**, 2, 4, 8, 9, 5, 10, 3, 6, 7

Infixe : 8, 4, 9, 2, 10, 5, **1**, 6, 3, 7

Suffixe : 8, 9, 4, 10, 5, 2, 6, 7, 3, **1**




```
fonction parcours_profondeur_recursif_prefixe(arbre):
        lst2=créer liste vide
        si nœud est une liste
                ajouter à la fin de lst2 le premier élément de nœud #parcours préfixe
                si fils droit non vide
                        parcours_profondeur_recursif_prefixe(fils droit)

                ajouter à la fin de lst2 le premier élément de nœud # parcours infixe

                si fils gauche non vide
                        parcours_profondeur_recursif_prefixe(fils gauche)
                ajouter à la fin de lst2 le premier élément de nœud # parcours suffixe ou postfixe
        sinon
                ajouter à la fin de lst2 nœud
        retourner lst2
```